Hands-On MLOps Workshop

Git + DVC + MLflow

A Step-by-Step Practical Guide

AI Bootcamp

Environment: VS Code with Ubuntu/iMac Terminal

Focus: MLOps Workflow — not model performance

Phase 1: Project Setup & Git

Step 1: Create Project Directory and Initialize Git

```
mkdir mlops-project
cd mlops-project
git init
```

Concept: git init creates a hidden .git/ folder inside your project. This is where git stores the entire history — every commit, every branch, every change. Without this, git has no idea your folder exists.

Why: In any MLOps project, version control is the foundation. Everything else — DVC, MLflow, your code — sits on top of git. We initialize it first because we want git to track the project from the very beginning. Git gives you a time machine. In ML projects this matters even more because you are constantly experimenting. 'The model worked better last Thursday' is a real sentence ML engineers say. Git lets you go back to last Thursday.

Step 2: Create a Virtual Environment

```
python3 -m venv venv
source venv/bin/activate
```

Concept: A virtual environment is an isolated Python installation. Packages you install here do not affect your system Python or other projects.

Why: Reproducibility starts here. If your teammate has scikit-learn 1.2 and you have 1.5, your model might produce different results on the same data. An isolated environment pins this down. In production MLOps, this evolves into Docker containers, but venv is the first building block. We use venv over conda because it is built into Python — zero extra installation.

Step 3: Create requirements.txt

Create a file called requirements.txt in the project root:

```
scikit-learn
pandas
numpy
dvc
mlflow
pyyaml
```

Then install:

```
pip install -r requirements.txt
```

Concept: requirements.txt is a manifest of your project's dependencies. Anyone can clone your repo and run pip install -r requirements.txt to get the exact same environment.

Why: If you install packages one by one without recording them, nobody (including future you) knows what the project needs. This is a common mistake in ML projects — the model works on your machine but fails elsewhere.

In production, you would pin exact versions like scikit-learn==1.5.0. We include mlflow now even though we will not use it yet — it is a project dependency we know we will need.

Step 4: Create the Project Structure

```
mkdir -p src data/raw data/processed models
```

Concept: This creates: src/ for all source code (preprocessing, training, evaluation modules), data/raw/ for original untouched data, data/processed/ for cleaned/transformed data ready for training, and models/ for saved trained model artifacts.

Why: data/raw and data/processed are separate because raw data is sacred — you never modify it. Your preprocessing script reads from raw, writes to processed. If preprocessing has a bug, raw data is still intact. The src/ directory keeps code modular — in production, your training code might be imported by a serving layer, a testing framework, or a pipeline orchestrator.

Step 5: Create .gitignore

Create .gitignore in the project root:

```
# Virtual environment
venv/

# Python
__pycache__/
*.pyc
*.pyo

# MLflow
mlruns/
mlartifacts/

# OS
.DS_Store

# IDE
.vscode/
```

Concept: .gitignore tells git 'pretend these files do not exist.' Git will never track, commit, or complain about them.

Why: This is where MLOps thinking begins. Code belongs in git (small, text-based). Data does NOT belong in git (large, binary, git cannot diff it) — DVC will handle this. Model files do NOT belong in git. MLflow runs do NOT belong in git. venv/ does NOT belong in git (recreatable from requirements.txt). The golden rule: Git is for things humans write, not for things machines generate. Important: Do NOT add data/ or models/ here — DVC will create its own .gitignore files inside those directories when it tracks them.

Step 6: Create params.yaml

Create params.yaml in the project root:

```
data:
  test_size: 0.2
```

```
  random_state: 42

train:
  model_type: "RandomForest"
  n_estimators: 100
  max_depth: 10
  random_state: 42
```

Concept: Config-driven approach. All hyperparameters live in one YAML file, not scattered as magic numbers inside your code.

Why: Later, three tools will read this single file — your code, DVC, and MLflow. Change one line, rerun the pipeline, everything stays in sync. DVC looks for params.yaml at the project root by default, so we follow the convention.

Phase 2: Write Modular Code

Step 7: Create src/preprocess.py

Create src/preprocess.py:

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
import yaml
import os


def load_params():
    with open("params.yaml", "r") as f:
        params = yaml.safe_load(f)
    return params


def preprocess():
    params = load_params()

    # Load wine dataset
    wine = load_wine()
    df = pd.DataFrame(wine.data, columns=wine.feature_names)
    df["target"] = wine.target

    # Save raw data
    os.makedirs("data/raw", exist_ok=True)
    df.to_csv("data/raw/wine.csv", index=False)

    # Split data
    train_df, test_df = train_test_split(
        df,
        test_size=params["data"]["test_size"],
        random_state=params["data"]["random_state"],
    )

    # Save processed data
    os.makedirs("data/processed", exist_ok=True)
    train_df.to_csv("data/processed/train.csv", index=False)
    test_df.to_csv("data/processed/test.csv", index=False)

    print(f"Data preprocessed: train={len(train_df)}, test={len(test_df)}")


if __name__ == "__main__":
    preprocess()
```

Concept: First pipeline stage — loads data, splits it, and saves the splits to disk.

Why: The test split size and random state are read from params.yaml, not hardcoded. Change the config, not the code.

Step 8: Create src/train.py

Create src/train.py:

```
import pandas as pd
import pickle
import yaml
import os
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression


def load_params():
    with open("params.yaml", "r") as f:
        params = yaml.safe_load(f)
    return params


def train():
    params = load_params()
    train_params = params["train"]

    # Load processed training data
    train_df = pd.read_csv("data/processed/train.csv")
    X_train = train_df.drop("target", axis=1)
    y_train = train_df["target"]

    # Select model based on config
    if train_params["model_type"] == "RandomForest":
        model = RandomForestClassifier(
            n_estimators=train_params["n_estimators"],
            max_depth=train_params["max_depth"],
            random_state=train_params["random_state"],
        )
    elif train_params["model_type"] == "LogisticRegression":
        model = LogisticRegression(
            max_iter=1000,
            random_state=train_params["random_state"],
        )
    else:
        raise ValueError(f"Unknown model type: {train_params['model_type']}")

    # Train
    model.fit(X_train, y_train)

    # Save model
    os.makedirs("models", exist_ok=True)
    with open("models/model.pkl", "wb") as f:
        pickle.dump(model, f)

    print(f"Model trained: {train_params['model_type']}")


if __name__ == "__main__":
    train()
```

Concept: Training stage — clean, simple, no tracking. Reads config, trains a model, saves a pickle file.

Why: Notice what is missing: no experiment tracking, no record of which parameters were used, no run ID. Just a pickle file that gets overwritten every time you train. We start without MLflow so you feel the pain first. After a few experiments, you will not know what params produced what results. That is when MLflow will click.

Step 9: Create src/evaluate.py

Create src/evaluate.py:

```
import pandas as pd
import pickle
import json
import os
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score


def evaluate():
    # Load test data
    test_df = pd.read_csv("data/processed/test.csv")
    X_test = test_df.drop("target", axis=1)
    y_test = test_df["target"]

    # Load model
    with open("models/model.pkl", "rb") as f:
        model = pickle.load(f)

    # Predict
    y_pred = model.predict(X_test)

    # Calculate metrics
    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "f1_score": f1_score(y_test, y_pred, average="weighted"),
        "precision": precision_score(y_test, y_pred, average="weighted"),
        "recall": recall_score(y_test, y_pred, average="weighted"),
    }

    # Save metrics locally
    os.makedirs("models", exist_ok=True)
    with open("models/metrics.json", "w") as f:
        json.dump(metrics, f, indent=4)

    print("Evaluation metrics:")
    for metric, value in metrics.items():
        print(f"  {metric}: {value:.4f}")


if __name__ == "__main__":
    evaluate()
```

Concept: Loads the trained model, runs predictions on test data, computes metrics, saves to JSON, and prints to terminal.

Why: Metrics print to the terminal and get saved to a single JSON file. Every run overwrites the previous metrics. After 5 experiments, you will be scrolling through terminal history trying to find 'what was the accuracy when I used LogisticRegression?' That is the problem MLflow solves.

Step 10: Run the Pipeline Manually and First Git Commit

```
python src/preprocess.py
python src/train.py
python src/evaluate.py
```

See the metrics printed. Then commit:

```
git add .gitignore params.yaml requirements.txt src/
git commit -m "Initial project structure with preprocessing, training, and evaluation
modules"
```

Concept: We run all three scripts manually to verify everything works end-to-end. Then we commit only code and config.

Why: Before adding any tooling (DVC, MLflow), verify the base code works. Debug code first, then layer tools on top. If something breaks later, you know it is the tool integration, not the code itself.

Phase 3: DVC — Data & Pipeline Versioning

Step 11: Initialize DVC

```
dvc init
```

Concept: Just like git init created .git/, dvc init creates a .dvc/ directory — DVC's brain. It also auto-stages some files in git.

Why: DVC does not replace git — it extends it. DVC stores pointers (small .dvc files) in git while the actual large files go to DVC's own storage. Think of it as git tracking a receipt that says 'your data is over there.'

Step 12: Set Up a Local DVC Remote

```
mkdir -p /tmp/dvc-remote
dvc remote add -d myremote /tmp/dvc-remote
```

Concept: A DVC remote is where actual data files get pushed to — like GitHub but for data. The -d flag makes it the default remote.

Why: In production, this would be S3, GCS, or Azure Blob. For now, a local folder works identically. The mental model: Git pushes code to GitHub. DVC pushes data to a remote. Two parallel systems, perfectly synced.

Step 13: Track Data with DVC

```
dvc add data/raw
dvc add data/processed
```

Concept: dvc add does four things: (1) computes a hash of the data, (2) moves data into DVC's local cache, (3) creates a .dvc pointer file for git, (4) adds the actual data to a .gitignore inside that directory.

Why: The .dvc file is tiny — just a hash. Git tracks it happily. The actual data (potentially gigabytes) lives in DVC's cache and remote. Teammates clone the repo, get .dvc files, then dvc pull to download data.

Step 14: Commit DVC Tracking Files

```
git add data/raw.dvc data/processed.dvc .gitignore
git commit -m "Track raw and processed data with DVC"
```

Concept: Git now knows 'at this commit, the data had this hash.' Checkout an older commit + dvc checkout = old data restored.

Why: Git versions code. DVC versions data. They are linked through .dvc pointer files. Time travel works for both simultaneously.

Step 15: Push Data to DVC Remote

```
dvc push
```

Concept: Uploads data from local DVC cache to the remote. Like git push for data.

Step 16: Create the DVC Pipeline (dvc.yaml)

Create dvc.yaml in the project root:

```
stages:
  preprocess:
    cmd: python src/preprocess.py
    deps:
      - src/preprocess.py
    params:
      - data.test_size
      - data.random_state
    outs:
      - data/raw
      - data/processed

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/processed
    params:
      - train
    outs:
      - models/model.pkl

  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
      - models/model.pkl
      - data/processed
    metrics:
      - models/metrics.json:
          cache: false
```

Concept: The pipeline definition — the heart of DVC. It describes your ML workflow as a DAG. Each stage has: cmd (command to run), deps (dependencies — if any change, stage reruns), params (parameters from params.yaml), outs (output files), and metrics (special outputs for metric tracking).

Why: Change n_estimators in params.yaml: DVC checks preprocess deps — unchanged, skip. Train params — changed, rerun. Evaluate deps — model changed, rerun. It only runs what is necessary.

Important — remove old .dvc files since pipeline now manages these outputs:

```
dvc remove data/raw.dvc
dvc remove data/processed.dvc
```

A file can be tracked by dvc add OR by dvc.yaml, but not both. The old .dvc files would create a conflict.

Step 17: Run the Pipeline

```
dvc repro
```

Concept: dvc repro (reproduce) runs the entire pipeline respecting the dependency graph. It also creates a dvc.lock file — a lockfile recording exact hashes of every dependency, parameter, and output.

Why: It respects the DAG (correct order), skips unchanged stages (saves time), updates dvc.lock (records exact state), and it is one command anyone can run.

Step 18: Commit the Pipeline

```
git add dvc.yaml dvc.lock .gitignore models/metrics.json
git commit -m "Add DVC pipeline with preprocessing, training, and evaluation stages"
dvc push
```

Step 19: Run a Second Experiment

Edit params.yaml — change model_type to LogisticRegression:

```
data:
  test_size: 0.2
  random_state: 42

train:
  model_type: "LogisticRegression"
  n_estimators: 100
  max_depth: 10
  random_state: 42

dvc repro
```

Concept: preprocess is SKIPPED (data params unchanged). train RERUNS (model_type changed). evaluate RERUNS (model changed).

```
git add params.yaml dvc.lock models/metrics.json
git commit -m "Experiment: LogisticRegression"
dvc push
```

Step 20: Run a Third Experiment

Edit params.yaml:

```
data:
  test_size: 0.2
  random_state: 42

train:
  model_type: "RandomForest"
  n_estimators: 200
  max_depth: 20
  random_state: 42

dvc repro
```

```
git add params.yaml dvc.lock models/metrics.json
git commit -m "Experiment: RandomForest n_estimators=200 max_depth=20"
dvc push
```

Why: The pain is now real: you have run 3 experiments. Which one had the best F1 score? What params did you use for each? You would need to git log, git show each commit's metrics.json and params.yaml, and manually compare. This is painful. This is exactly why MLflow exists.

Phase 4: Introducing MLflow — Step by Step

Step 21: What is MLflow? Introduction

Before touching any code, let us understand what MLflow gives us.

MLflow has four main components:

1. MLflow Tracking — logs parameters, metrics, and artifacts for each run

2. MLflow Projects — packages code for reproducibility (DVC handles this for us)

3. MLflow Models — standard format for packaging ML models

4. MLflow Model Registry — version control for models (staging, production, archiving)

Key MLflow vocabulary:

Experiment — a named group of related runs (like a project folder). Example: 'wine-classification'

Run — a single execution of training. Each run records its params, metrics, and artifacts

Parameter — an input setting (n_estimators=100)

Metric — an output measurement (accuracy=0.95)

Artifact — an output file (model.pkl, plots, etc.)

Step 22: First MLflow Touch — Create an Experiment and Start a Run

Edit src/train.py — add ONLY the experiment and run structure. The full updated file:

```
import pandas as pd
import pickle
import yaml
import os
import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression


def load_params():
    with open("params.yaml", "r") as f:
        params = yaml.safe_load(f)
    return params


def train():
    params = load_params()
    train_params = params["train"]

    # Load processed training data
    train_df = pd.read_csv("data/processed/train.csv")
```

```
    X_train = train_df.drop("target", axis=1)
    y_train = train_df["target"]

    # Select model based on config
    if train_params["model_type"] == "RandomForest":
        model = RandomForestClassifier(
            n_estimators=train_params["n_estimators"],
            max_depth=train_params["max_depth"],
            random_state=train_params["random_state"],
        )
    elif train_params["model_type"] == "LogisticRegression":
        model = LogisticRegression(
            max_iter=1000,
            random_state=train_params["random_state"],
        )
    else:
        raise ValueError(f"Unknown model type: {train_params['model_type']}")

    # NEW: Set MLflow experiment
    mlflow.set_experiment("wine-classification")

    # NEW: Start an MLflow run
    with mlflow.start_run():
        # Train
        model.fit(X_train, y_train)

        # Save model
        os.makedirs("models", exist_ok=True)
        with open("models/model.pkl", "wb") as f:
            pickle.dump(model, f)

        print(f"Model trained: {train_params['model_type']}")
        print(f"MLflow Run ID: {mlflow.active_run().info.run_id}")


if __name__ == "__main__":
    train()
```

What changed (only 3 lines):

1. import mlflow

2. mlflow.set_experiment("wine-classification") — create/select a named experiment

3. with mlflow.start_run(): — wrap training in a tracked run

> **Concept:** set_experiment creates a named bucket for your runs. start_run() opens a tracking context. Everything inside the with block is one 'run.' MLflow automatically creates a mlruns/ directory to store run data locally. Right now we are not logging anything — just establishing the structure.

Run it:

```
dvc repro
```

Launch the MLflow UI:

```
mlflow ui
```

Open http://127.0.0.1:5000 in your browser.

Why: You see an experiment called 'wine-classification' with one run. Click it — it has a Run ID, start time, and status, but NO parameters and NO metrics. It is an empty shell. Stop the server with Ctrl+C.

```
git add src/train.py
git commit -m "Introduce MLflow experiment and run tracking structure"
```

Step 23: Add Parameter Logging

Edit src/train.py — add mlflow.log_params() inside the run. Find the section inside 'with mlflow.start_run():' and add BEFORE the training:

```
        # NEW: Log parameters to MLflow
        mlflow.log_params(train_params)
        mlflow.log_param("test_size", params["data"]["test_size"])

        # Train
        model.fit(X_train, y_train)
```

What changed (2 lines):

1. mlflow.log_params(train_params) — logs the entire train config dict in one call

2. mlflow.log_param("test_size", ...) — logs a single parameter from the data config

Concept: log_params() takes a dictionary and logs all key-value pairs. log_param() logs a single key-value pair. Parameters are INPUT settings — things you decided BEFORE training.

```
dvc repro
mlflow ui
```

Check the UI — the Parameters section is now populated. Stop with Ctrl+C.

```
git add src/train.py
git commit -m "Add MLflow parameter logging to training"
```

Step 24: Add Metric Logging

Edit src/evaluate.py — add MLflow metric logging. The full updated file:

```
import pandas as pd
import pickle
import json
import os
import mlflow
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score


def evaluate():
    # Load test data
    test_df = pd.read_csv("data/processed/test.csv")
    X_test = test_df.drop("target", axis=1)
    y_test = test_df["target"]

    # Load model
    with open("models/model.pkl", "rb") as f:
        model = pickle.load(f)

    # Predict
    y_pred = model.predict(X_test)

    # Calculate metrics
    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
```

```
        "f1_score": f1_score(y_test, y_pred, average="weighted"),
        "precision": precision_score(y_test, y_pred, average="weighted"),
        "recall": recall_score(y_test, y_pred, average="weighted"),
    }

    # Save metrics locally (for DVC)
    os.makedirs("models", exist_ok=True)
    with open("models/metrics.json", "w") as f:
        json.dump(metrics, f, indent=4)

    # NEW: Log metrics to MLflow
    experiment = mlflow.get_experiment_by_name("wine-classification")
    if experiment:
        runs = mlflow.search_runs(
            experiment_ids=[experiment.experiment_id],
            order_by=["start_time DESC"],
            max_results=1,
        )
        if not runs.empty:
            run_id = runs.iloc[0]["run_id"]
            with mlflow.start_run(run_id=run_id):
                mlflow.log_metrics(metrics)

    print("Evaluation metrics:")
    for metric, value in metrics.items():
        print(f"  {metric}: {value:.4f}")


if __name__ == "__main__":
    evaluate()
```

Concept: Training and evaluation are separate scripts run at different times. But logically, they belong to the SAME run. So evaluate finds the most recent run (the one train.py created), reopens it, and appends metrics. One run = one complete train+evaluate cycle.

Why: We keep the JSON file too: models/metrics.json is for DVC (dvc metrics diff). MLflow metrics are for the UI. Different tools, different purposes. DVC handles pipeline reproducibility. MLflow handles experiment comparison.

```
dvc repro
mlflow ui
```

Check the UI — both Parameters AND Metrics are populated. Stop with Ctrl+C.

```
git add src/evaluate.py
git commit -m "Add MLflow metric logging to evaluation"
```

Step 25: Add Artifact Logging

Edit src/train.py — after saving the model pickle, add:

```
        # NEW: Log model artifact to MLflow
        mlflow.log_artifact("models/model.pkl")
```

Edit src/evaluate.py — inside the with mlflow.start_run(run_id=run_id): block, after mlflow.log_metrics(metrics), add:

```
                mlflow.log_artifact("models/metrics.json")
```

Concept: Artifacts are OUTPUT files. MLflow copies them into its own storage (mlruns/), associated with the specific run. Now each run has: what params you used, what metrics you got, AND the actual model file.

```
dvc repro
mlflow ui
```

Click the latest run, go to the Artifacts tab — model.pkl and metrics.json are stored there. Stop with Ctrl+C.

```
git add src/train.py src/evaluate.py
git commit -m "Add MLflow artifact logging for model and metrics"
```

Step 26: Run Multiple Experiments and Compare

Experiment B — LogisticRegression:

Edit params.yaml:

```
data:
  test_size: 0.2
  random_state: 42

train:
  model_type: "LogisticRegression"
  n_estimators: 100
  max_depth: 10
  random_state: 42

dvc repro
```

Experiment C — RandomForest with fewer estimators:

Edit params.yaml:

```
data:
  test_size: 0.2
  random_state: 42

train:
  model_type: "RandomForest"
  n_estimators: 50
  max_depth: 5
  random_state: 42

dvc repro
```

Compare them all:

```
mlflow ui
```

Open http://127.0.0.1:5000. Select all runs, click Compare. You get a side-by-side comparison — which model, which params, which metrics.

> **Why:** Remember when you ran 3 experiments without MLflow and could not compare them? Now you have a dashboard showing everything at a glance. You can answer 'which was best?' in seconds.

Also try in terminal:

```
git add params.yaml dvc.lock models/metrics.json
git commit -m "Run multiple experiments with MLflow tracking"
dvc push
```

Phase 5: MLflow Model Logging, Registry & Serving

Step 27: Log the Model Using MLflow's Model Format

Currently we save the model as a raw pickle file. MLflow has its own model format that adds metadata, dependencies, and a standard interface.

Edit src/train.py — find the pickle save + artifact log section inside with mlflow.start_run(): and replace it with:

```
        # Save model as pickle (for DVC pipeline)
        os.makedirs("models", exist_ok=True)
        with open("models/model.pkl", "wb") as f:
            pickle.dump(model, f)

        # NEW: Log model using MLflow's sklearn integration
        mlflow.sklearn.log_model(
            sk_model=model,
            artifact_path="wine-model",
            input_example=X_train.iloc[:1],
        )
```

Concept: mlflow.sklearn.log_model() saves the model in MLflow's standard format. It automatically records: what library was used, what version, what dependencies are needed. artifact_path names this artifact group. input_example saves a sample input row so anyone loading the model later knows the expected data shape.

Why: A raw pickle file is fragile — it requires the exact same Python version, sklearn version, and environment. MLflow's model format records all of this, making the model portable. This is also a prerequisite for the Model Registry.

```
dvc repro
mlflow ui
```

Click the latest run, go to Artifacts tab — you will see a wine-model/ folder containing model.pkl, MLmodel, conda.yaml, requirements.txt, and input_example.json. Stop with Ctrl+C.

```
git add src/train.py
git commit -m "Log model using MLflow sklearn integration with input example"
```

Step 28: Register the Model in MLflow Model Registry

The Model Registry is MLflow's version control for models — name, version, and stage management.

Edit src/train.py — add the registered_model_name parameter to log_model. Find mlflow.sklearn.log_model and add:

```
        mlflow.sklearn.log_model(
            sk_model=model,
            artifact_path="wine-model",
            input_example=X_train.iloc[:1],
            registered_model_name="WineClassifier",
        )
```

Concept: This single parameter means every time you train, MLflow: (1) Logs the model as before. (2) Checks the registry: does 'WineClassifier' exist? If no, creates it as Version 1. If yes, creates a new version (Version 2, 3, etc.)

Set params.yaml to RandomForest with n_estimators=100, max_depth=10 and run:

```
dvc repro
mlflow ui
```

Click 'Models' in the top navigation bar. You will see 'WineClassifier' Version 1. Stop with Ctrl+C.

Step 29: Create Multiple Model Versions

Train Version 2:

Edit params.yaml: model_type=RandomForest, n_estimators=200, max_depth=15

```
dvc repro
```

Train Version 3:

Edit params.yaml: model_type=LogisticRegression

```
dvc repro
mlflow ui
```

Go to Models > WineClassifier. You now see Version 1, 2, and 3. Each version links back to the exact run that produced it — full traceability. Stop with Ctrl+C.

```
git add src/train.py params.yaml dvc.lock models/metrics.json
git commit -m "Add MLflow model registry with multiple versions"
dvc push
```

Step 30: Assign Aliases to Model Versions

MLflow uses aliases to tag which model version serves which purpose. Common aliases: champion (production) and challenger (being tested).

Create src/register_model.py:

```
from mlflow import MlflowClient


def register():
    client = MlflowClient()

    # Set alias 'champion' to version 2
    client.set_registered_model_alias(
        name="WineClassifier",
        alias="champion",
        version=2,
    )

    # Set alias 'challenger' to version 3
    client.set_registered_model_alias(
        name="WineClassifier",
        alias="challenger",
        version=3,
    )

    # Verify
    champion = client.get_model_version_by_alias("WineClassifier", "champion")
    challenger = client.get_model_version_by_alias("WineClassifier", "challenger")

    print(f"Champion: Version {champion.version}")
    print(f"Challenger: Version {challenger.version}")
```

```
if __name__ == "__main__":
    register()

python src/register_model.py
```

Concept: Aliases are tags you attach to model versions. When Version 4 proves itself better, you move the 'champion' alias. No code change needed in your serving layer — it always loads the 'champion' alias. MlflowClient is the Python API for interacting with the MLflow tracking server programmatically.

Why: In production, your serving code says 'load the champion model.' When the data science team promotes a new version, they move the alias. The serving code does not change. This decouples model development from model deployment.

```
git add src/register_model.py
git commit -m "Add model alias assignment script for champion and challenger"
```

Step 31: Load a Registered Model for Prediction

Now the payoff — using a registered model to make predictions. Create src/predict.py:

```
import pandas as pd
import mlflow


def predict():
    # Load the champion model from the registry
    model_uri = "models:/WineClassifier@champion"
    model = mlflow.sklearn.load_model(model_uri)

    print(f"Loaded model from: {model_uri}")
    print(f"Model type: {type(model).__name__}")

    # Load test data for demonstration
    test_df = pd.read_csv("data/processed/test.csv")
    X_test = test_df.drop("target", axis=1)
    y_test = test_df["target"]

    # Make predictions
    predictions = model.predict(X_test)

    # Show results
    results = pd.DataFrame({
        "actual": y_test.values,
        "predicted": predictions,
    })
    results["correct"] = results["actual"] == results["predicted"]

    print(f"\nPredictions on test set:")
    print(results.head(10))
    print(f"\nAccuracy: {results['correct'].mean():.4f}")
    print(f"Total samples: {len(results)}")


if __name__ == "__main__":
    predict()
```

```
python src/predict.py
```

Concept: models:/WineClassifier@champion is an MLflow model URI. models:/ tells MLflow to look in the registry. WineClassifier is the model name. @champion is the alias to load. Other formats: models:/WineClassifier/2 (specific version), runs:/<run_id>/wine-model (from a specific run directly).

Why: Your prediction code never mentions a file path, pickle, or sklearn version. If the team promotes a new champion, this code automatically uses it. MLflow handles deserialization, dependency checking, and the model interface.

```
git add src/predict.py
git commit -m "Add prediction script using MLflow model registry"
```

Step 32: Compare Champion vs Challenger Programmatically

Create src/compare_models.py:

```
import pandas as pd
import mlflow
from sklearn.metrics import accuracy_score, f1_score


def compare():
    # Load test data
    test_df = pd.read_csv("data/processed/test.csv")
    X_test = test_df.drop("target", axis=1)
    y_test = test_df["target"]

    # Load both models
    champion = mlflow.sklearn.load_model("models:/WineClassifier@champion")
    challenger = mlflow.sklearn.load_model("models:/WineClassifier@challenger")

    # Predict with both
    champion_preds = champion.predict(X_test)
    challenger_preds = challenger.predict(X_test)

    # Compare
    results = {
        "Model": ["Champion", "Challenger"],
        "Type": [type(champion).__name__, type(challenger).__name__],
        "Accuracy": [
            accuracy_score(y_test, champion_preds),
            accuracy_score(y_test, challenger_preds),
        ],
        "F1 Score": [
            f1_score(y_test, champion_preds, average="weighted"),
            f1_score(y_test, challenger_preds, average="weighted"),
        ],
    }

    df = pd.DataFrame(results)
    print("Champion vs Challenger Comparison:")
    print(df.to_string(index=False))

    # Determine winner
    if results["F1 Score"][1] > results["F1 Score"][0]:
        print("\nChallenger outperforms Champion! Consider promoting.")
    else:
        print("\nChampion still holds. Challenger needs more work.")


if __name__ == "__main__":
    compare()
```

```
python src/compare_models.py
```

Concept: This is a real-world pattern — automated model comparison. In production CI/CD, this script would run automatically whenever a new challenger is registered, and promote it to champion if it passes quality gates.

```
git add src/compare_models.py
git commit -m "Add champion vs challenger comparison script"
```

Phase 6: The Big Picture

Step 33: View the Pipeline DAG

```
dvc dag
```

Concept: Prints your pipeline as a text-based DAG: preprocess > train > evaluate. As projects grow, this visualization helps everyone understand the flow.

Step 34: Final Commit and Review

```
git log --oneline
```

Your git history tells the complete story — from plain code, to DVC, to MLflow piece by piece, to model registry and serving. The learning journey is visible in the commits.

Step 35: Final Project Structure

```
mlops-project/
├── src/
│   ├── preprocess.py          # Data loading and splitting
│   ├── train.py               # Model training + MLflow logging
│   ├── evaluate.py            # Model evaluation + MLflow metrics
│   ├── predict.py             # Load registered model for inference
│   ├── compare_models.py      # Champion vs challenger comparison
│   └── register_model.py      # Assign model aliases
├── data/
│   ├── raw/                   # Tracked by DVC
│   └── processed/             # Tracked by DVC
├── models/
│   ├── model.pkl              # Tracked by DVC
│   └── metrics.json           # Tracked by DVC + MLflow
├── mlruns/                    # MLflow's tracking data
├── params.yaml                # Single source of truth for config
├── dvc.yaml                   # Pipeline definition
├── dvc.lock                   # Pipeline state lock
├── requirements.txt           # Python dependencies
└── .gitignore                 # What git ignores
```

Summary: Who Does What

Concern	Tool	What It Tracks
Code & config versioning	**Git**	.py files, params.yaml, dvc.yaml, dvc.lock
Data & model versioning	**DVC**	Datasets, trained models, pipeline outputs
Experiment tracking	**MLflow Tracking**	Parameters, metrics, artifacts per run
Model versioning	**MLflow Registry**	Named models, versions, aliases

Model serving	**MLflow Models**	Standardized model format, load by URI

The Mental Model

Git answers: 'What did the code look like at this point?'

DVC answers: 'What did the data and model look like at this point?'

MLflow Tracking answers: 'What happened when I ran the code on the data?'

MLflow Registry answers: 'Which model version is in production right now?'

Together, they give you complete reproducibility, traceability, and deployment readiness across your entire ML workflow.